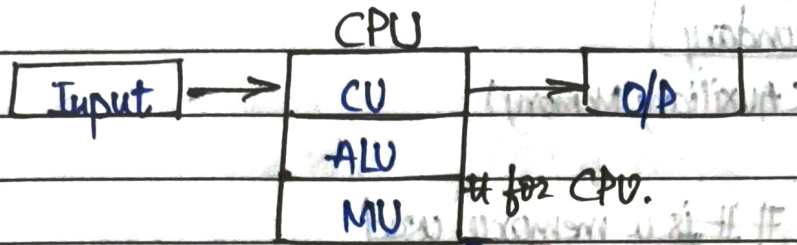


27/9/26

Page No.

Date: / /

Computer: It is an electronic device which performs millions of operations in a fraction of time with 100% accuracy.



Secondary memory # for us

CPU: Central Processing Unit

CU: Control unit

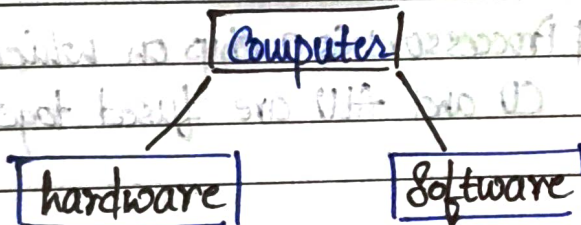
ALU: Arithmetic Logical Unit

MU: Memory Unit

Output on screen:

soft copy output

Output in form of printout:
hardcopy output



Program: Set of instructions.

all the tangible parts of computer which can be seen or touched is keyboard, mouse etc.

set of programmes which makes hardware part operational.

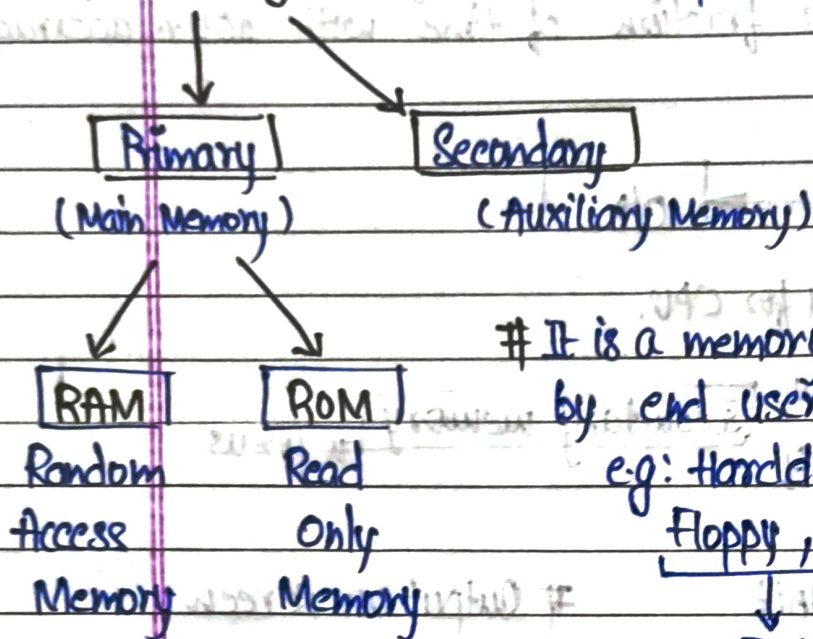
System Soft.

Application Soft.

The set of programmes which are essential for functioning of a system, these are generalised programs. eg: OS (windows)

These are the programmes which are specific to a particular need. eg MS Office, Chrome, Instagram etc.

Memory: Place where a computer can store data.



It is a memory used by end user.

e.g: Harddisk, Floppy, CD etc.

↓
Tertiary (also sec.)

The memory which is used by CPU for its processing

Processor: is a chip on which CU and ALU are fused together.

1) RAM: It is a volatile memory and is used as active working area of computer.

2) ROM: It is a chip on which system instructions are (Booting already embedded. It is prog.) also known as **Firmware**.

Programming :

Language

Syntax

(Grammar rule)

Semantics

(Set of predefined words)

Levels of languages:

- High Level (for user/user friendly : like english)

- Assembly (hardware language)

- Binary

→ C is a middle level language

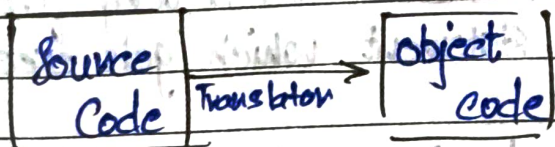
→ It is a language understood by computer and it is a language of zeroes and ones.

→ language which uses Mnemonic codes not user friendly. (Low level language)

→ Bit: Smallest unit of storage either 0 (zero) or 1 (one) is a bit.

1 Byte = 8 bits.

⇒ Translator: The program which converts one language program to another language.



1. **Compiler:** It translates or converts a high level language programs to machine language in one go.
 2. **Interpreter:** The program which converts a high level program to machine language statement by statement.
 3. **Assembler:** The program which converts assembly code to binary code.
- # **C language:** UNIX was written in C language.
- C is a structured programming language and procedural language.
 - It follows top-down approach.

STRUCTURE

```
#include <iostream>    C++
```

```
#include <stdio.h>      C
```

```
int main()
```

```
{
```

```
    _____
```

```
    _____
```

```
    _____
```

```
return 0;
```

```
}
```

```
#include
```

Preprocessor
directive

stdio.h → standard input output

↳ header file / library file.

int main → it is a very first statement which gets executed.

→ 1) in front of anything is a funcⁿ. # main(): user-defined funcⁿ.

$$a = \frac{b+c+d}{3}$$

ST-PART

ST-PART
100

Page No. ay b

Date: 4x2

$$P = 2(y+z)$$

Write a program (WAP) (WAP); to input 2 no's, find their sum and print the result.

WAP to find S.I.

WAP to find Area and perimeter of a rectangle.

WAP to find Area of:
a) Isosceles Δ , Right angled Δ
b) Scalene Δ .

WAP to find avg marks of 3 subjects of every student.

#include <stdio.h>
int main() {

"Giving message is not necessary its desirable"

int a, b, c; $\rightarrow$ declaration statement.

printf("Enter 2 no's:"); $\rightarrow$ output funcⁿ.

scanf("%d %d", &a, &b); $\rightarrow$ Input funcⁿ through keyboard.

c = a + b;

printf("Sum is %d", c);

return 0;

}

let us C - ~~Rish~~ Yashwant Kounthran

$$\begin{aligned} & (b-3)(b-4)(b-5) \\ & b \times 3 \times 2 \times 1 \times b \\ & \int \frac{1}{x^b} \end{aligned}$$

$$\begin{aligned} & 16 \times 11 \times 11 \times 11 \times 11 \times 11 \\ & 16 \times 11 \times 11 \times 11 \times 11 \times 11 \end{aligned}$$

① #include <stdio.h>

#include <math.h>

int main()

{

float a,b,c;

printf("Enter two numbers:");

scanf("%f %f", &a, &b);

c = a + b;

printf("The sum is %f", c);

return 0;

}

②

#include <stdio.h>

#include <math.h>

int main()

{

float a,b,c,d;

printf("Enter principal, rate and time:");

scanf("%f %f %f", &a, &b, &c);

d = a * b * c / 100;

printf("The SI is: %f", d);

return 0;

}

③

```
#include <stdio.h>

int main() {
    float x, y, z, p;
    printf("Enter length and Breadth:");
    scanf("%f %f", &y, &z);
    x = y * z;
    p = 2(y + z);
    printf("The area of rectangle is %f and the perimeter is %f, x, p);

    return 0;
}
```

④

```
#include <stdio.h>

int main()
{
    float a, b, c, d;
    printf("Enter your marks:");
    scanf("%f %f %f", &a, &b, &c);
    d = (a + b + c) / 3;
    printf("The avg of your marks is : %f, d);

    return 0;
}
```

⑤

```
#include <stdio.h>
#include <math.h>
int main()
{
    float s, a, b, c, p;
    printf("Enter 3 sides of Triangle: ");
    scanf("%f %f %f", &a, &b, &c);
    s = (a + b + c) / 2;
```

```
p = sqrt(s*(s-a)*(s-b)*(s-c));  
printf("The area of scalene Triangle is : %f", p);
```

```
return 0;
```

```
}
```

Format specifiers:

%d = integer

%f = float

%c = character

%u = unsigned

%ld = long integer

→ Sizeof (): It is a library which tells the amount of memory occupied by a variable or any program element.

C-Tokens

↳ Identifier: Name given to any variable.

↳ Keywords: These are reserved words which have got predefined meaning (32).

↳ Constant: The tokens whose value is fixed. eg. const float

↳ operators: Symbols which tell the compiler to perform any operation. pi = 3.14

↳ special symbols: (), {}, []

Variables: Name given to any memory location which hold
datatype variable name ; some value

inta;
float b;
char c;

Rules:- It should start with an alphabet.

- It can be a combination of alphabet and digits.

No special symbol is allowed except (-).

Declaration statement: int a;
float sum();

Similarly,

char name[26];

→ initialisation statement: int x=32;

Algorithm: The step by step procedure to solve any problem.

①

1) Start

2) Input two no's a and b;

3) $c = a + b$

4) print c

5) Stop

②

1) Start

2) Input P, R, T;

3) $SI = (P \times R \times T) / 100$;

4) Print SI;

5) Stop

④

1) Start

2) Input marks a, b, c;

3) $Av = (a + b + c) / 3$

4) Print Av;

5) Stop

⑧

1) Start

2) Input L, B;

3) $A = L \times B$;4) $P = 2 \times (L + B)$

5) Print A, P;

6) Stop

⑤

1) Start

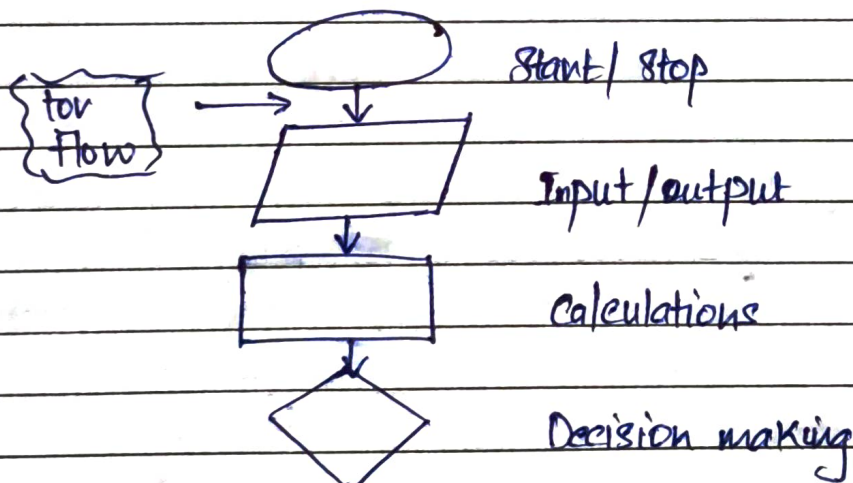
2) Input sides, p, q, r;

3) $s = (p + q + r) / 2$ 4) $Ar = \sqrt{s(s-a)(s-b)(s-c)}$

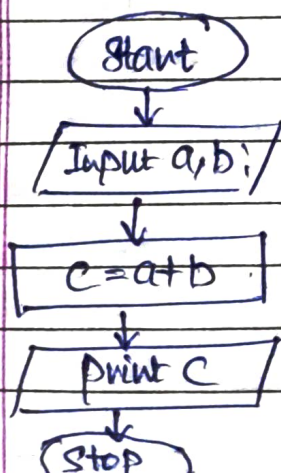
5) Print Ar;

6) Stop

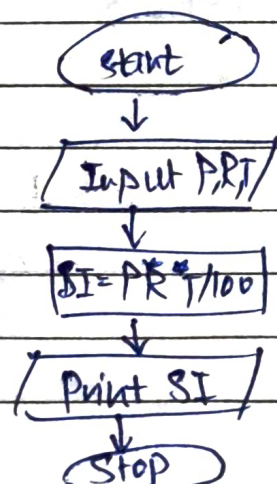
Flowchart: The pictorial representation of algorithm.



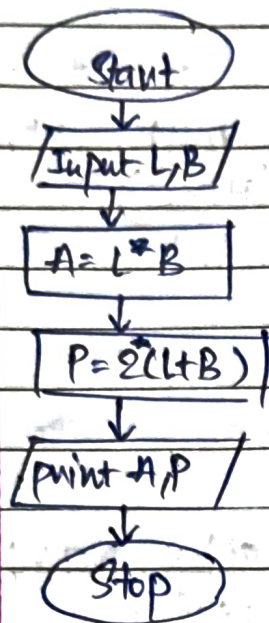
①



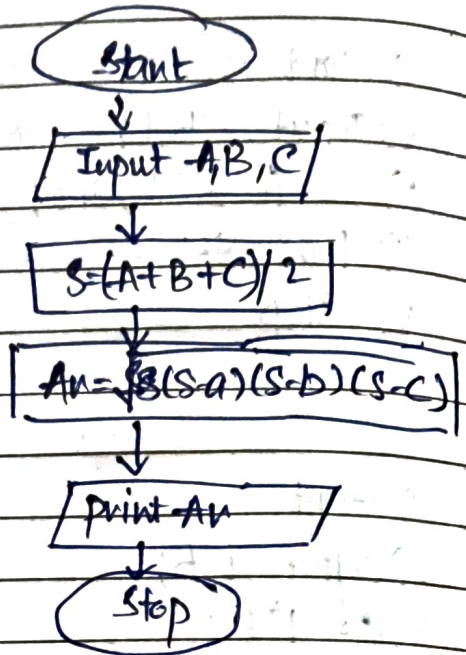
②



③



④



Operators in C: Operators are the symbols that performs any operation on operands.

→ Types of operators:

1) Arithmetic Operators: $+$, $-$, $*$, $/$ (division) quotient
 $\%$ (division) remainder

2) Relational Operators: $<$, $>$, $=$, $!=$, $<=$, $>=$.

3) Logical Operators: $\&$ (and)

$\|$ (or)

$!$ (not)

4) Assignment Operators: $=$ (right to left)

$=$: assignment operator
 $==$: comparison operator

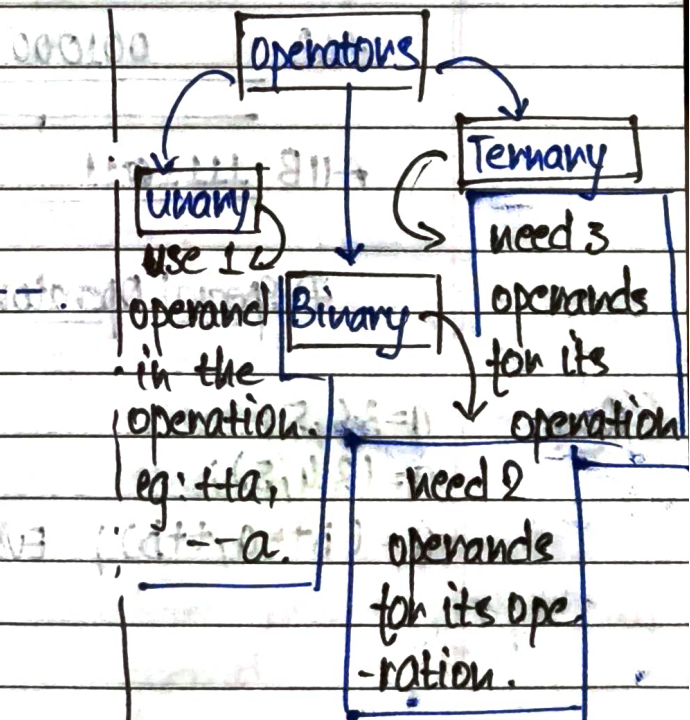
Increment and decrement: $++$, $--$.

Pre (++a)

incr. value of a by 1 then use the incr. value in the expression.

Post (a++)

use the current value of a in the expression then incr. by 1.



5) Conditional Operators: Conditional operators are ternary operators.

Generalised
Syntax:

var = exp1 ? exp2 : exp3

— ? — : —

→ if then else

→ (a < b) ? 1 : 0

6) Bitwise Operator: &

~ (complement)

<< (left shift)

>> (right shift)

eg: A = 101100

B = 011011

A & B

001000

A || B 1111011

7) Special Operators: ++, sizeof(), pointer, address.

Ques:

a = 3, 4, 5;

a = (3, 4, 5);

a = (b++ = a, ++b); Evaluate them.

Pseudocode: It is informal language independent representation of algorithm. to solve any problem, it is structured but does not follow any programming syntax.

Problem $\rightarrow$ algo. $\rightarrow$ Pseudocode $\rightarrow$ Program

START

Declare a, b, sum

Read a

Read b

Sum = a + b

Display Sum

STOP

Start

Input a, b

s = a + b

prints s

stop

Keywords: START/STOP

END

READ/INPUT

DISPLAY/OUTPUT

SET

Difference b/w algorithm and pseudocode:



steps

- It focuses on what needs to be performed.
- It uses english like sentences.
- More general.
- follows a structure but algo.
- what to do.



- It shows steps in a form closer to actual code.
- It uses special keywords.
- More detailed and program oriented.
- follows a programming like structure but no fixed syntax
- how to express those steps in code-like form.

START

Declare n

If $(n \% 2 == 0)$

OUTPUT n is even

Else

OUTPUT n is odd

STOP

Swap n/s: #include <stdio.h>

int main()

{

int a,b,c;

printf("Enter a,b,c");

scanf("%d %d",&a,&b);

c=a;

a=b;

b=c;

printf("The swap of values are %d %d",a,b)

return 0;

}

using
third
variable

Pseudocode: Start

Declare a,b,c

Read a

Read b

c=a

a=b

b=c

OUTPUT a,b

Stop

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b
```

```
    printf("Enter a, b");
```

```
    scanf("%d %d", &a, &b);
```

```
    a = a + b;
```

```
    b = a - b;
```

```
    a = a - b;
```

```
}
```

Dry run:

a = 10

b = 35

a = 10 + 35 = 45

b = 45 - 35 = 10

a = 45 - 10 = 35

Ques Char x, y

int s;

x = 'a'

y = 'b'

s = x + y

→

ASCII value: Default value

To find ASCII value of any character.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char a;
```

```
    printf("Enter a character"),
```

```
    scanf("%c", &a);
```

```
    printf("ASCII value in %d", a);
```

```
    return 0;
```

```
}
```

Write equivalent C expression:

i) $(m+n) * (a+b)$

ii.) $3x^2 + 2x + 5 : (3 * x * x) + 2 * x + 5$

iii) $\frac{a+b+c}{d+c} : (a+b+c)/(d+c)$

iv.) $\frac{2By}{d+1} - \frac{x}{3(z+y)} : ((2 * B * y)/(d+1) - x/(3 * (z+y)))$

v.) $z = \frac{(x+3)x^3}{(y-4)(y+5)} : ((x+3) * x * x * x)/((y-4) * (y+5))$

vi.) $A = \frac{776(xy+a)}{c} - 0.8 + 2b$

$((776 * (x * y + a))/c - (0.8 + 2 * b)) / ((x+a) * (1/y))$

% → Remainder
/ → Quotient

Ques

#include <stdio.h>

2/9

2

0.6

int main()

{

int i=2, j=3, k, l;

float a, b;

k = i / j * j;

l = j / i * i;

a = i / j * j;

b = j / i * i;

printf("%d %d %f %f\n", k, l, a, b);

return 0;

calculations.

→

$$\frac{2}{9}, \frac{3}{4}, \frac{2}{9}, \frac{3}{4}$$

$$0, 0, 0.22, 0.75$$

$$0, 0, 0, 2$$

$$0, 2, 1.8, 3$$

Ques

int main()

{

int a, b;

a = -3 - -25; +22

b = -3 - -(-3); -7

printf("a = %d b = %d\n", a, b);

return 0;

}

=

Reminders: Binary

- Ques: `int a=5, b=2;`
`float c=`
`(float)(a/b) + a/b`

Ques. Double X = 5.75
int total " (a)
(int)X * 2 + (int)(X*2)

Ques ~~char member =~~
char marker = 'A'
int f = marker + 57

Binary: Decimal $\rightarrow 0-9$
 base $\rightarrow 10$
 octal $\rightarrow 0-7 \quad ()_8$
 hexadecimal $\rightarrow 0-15 \quad ()_{16}$

Binary	Decimal
00	0
01	1
10	2
11	3

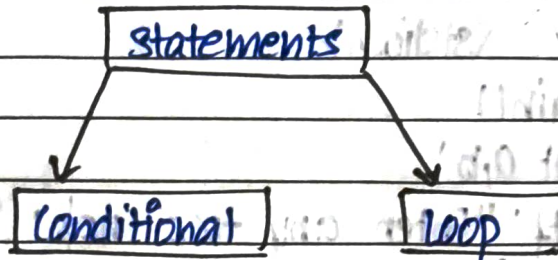
Conditional Statements:

if

if-else

nested-if

switch



Ques. WAP to check even or odd.

#include <stdio.h>

int main()

int a;

~~#~~

printf("Enter a no. : ");

scanf("%d", &a);

if

a%2==0

if a%2==0

{ printf("This is an even no."); }

→ change: else

return 0;

printf("No is odd");

→ Syntax of if: if (condition)

{

≡

}

else

{

≡

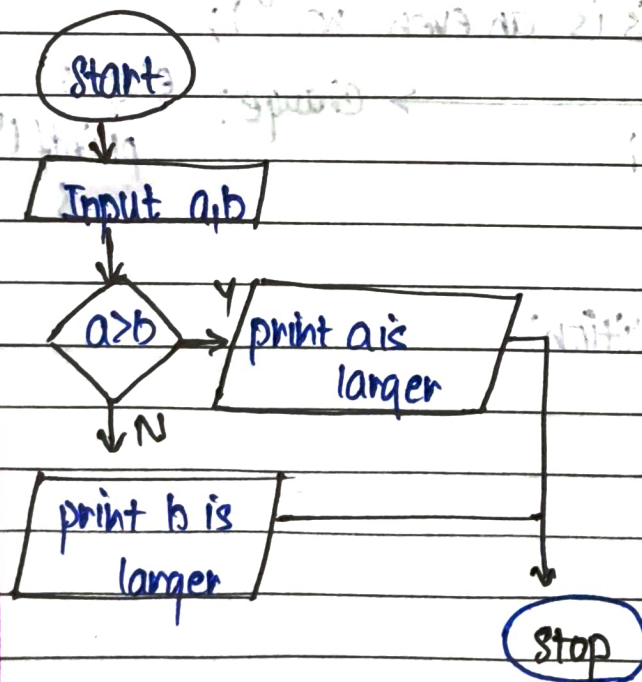
}

Ques WAP to find larger among two no's!

```
#include <stdio.h>
int main()
{
    int a, b;
    printf("Enter any two no's: ");
    scanf("%d %d", &a, &b);
    if (a > b)
    {
        printf("The larger no. is a");
    }
    else
    {
        printf("The larger no. is b");
    }
    return 0;
}
```

To replace this whole,

use (conditional operator) : $(a > b) ? \text{printf}("a \text{ is larger}");$
 also (Ternary operator) $\text{printf}("b \text{ is larger}");$



Start

Declare a, b

if $(a > b)$

output a is larger

else

output b is larger

STOP.

→ nested if...

Syntax: if (condition)

{

if (condition)

{

}

Ques WAP to find largest among 3 no's.

→ #include <stdio.h>

int main()

int a, b, c;

printf("Enter any three no's: ");

scanf("%d %d %d", &a, &b, &c);

if (a > b)

{

if (a > c)

{

printf("a is largest");

}

}

else if (b > a)

{

if (b > c)

{

printf("b is largest");

}

}

else {

printf("c is largest");

}

return 0;

}

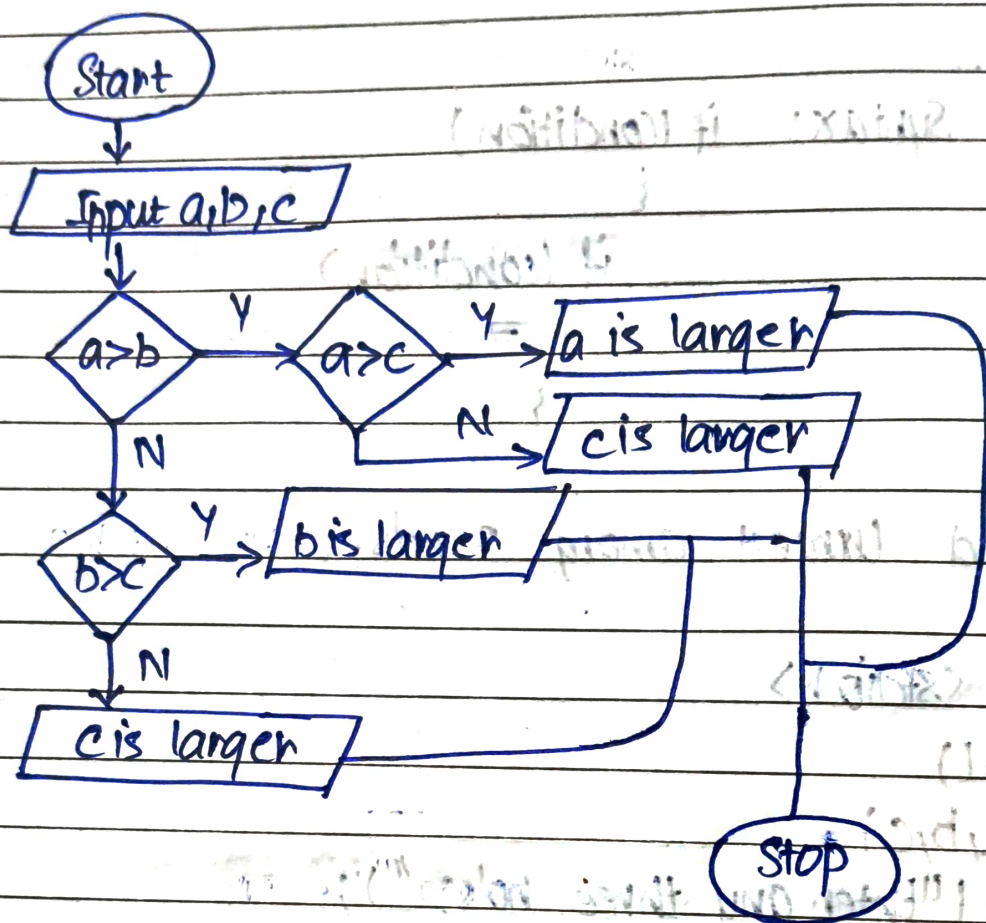
Chute
waate
if ka
else

else {

printf("b is largest");

}

"bade waate
if ka else."



Switch Statement:

Switch (expression)

{

case 'value':

{

====

break;

}

case 'value':

{

====

break;

}

default:

{

====

break;

{

}

default: {

printf("Wrong Choice");

break;

};

return 0;

{

}

#include <stdio.h>

int main() {

int a, b, c;

char ch;

printf("Enter two No's a, b and Choice");

scanf("%d %d %c", &a, &b, &c);

switch (ch)

{

case A: {

c = a + b;

printf("The sum of two no's is %d", c);

break;

}

case S: {

c = a - b;

printf("The subst of two no's is %d", c);

break;

}

case M: {

c = a * b;

printf("The multiplicaⁿ of two no's is %d", c);

break;

}

case D: {

c = a / b;

printf("The divⁿ of two no's is %d", c);

break;

}

WAP to print Day of the week acc to choice.

```
#include <stdio.h>
```

```
int main()
```

```
printf("Enter your no. ");
```

```
scanf("%d", &a);
```

```
switch (a);
```

Case 1:

```
{
    if (a == 1)
    {
        printf("Monday");
    }
}
```

```
break;
printf("Monday");
break;
}
```

Case 2:

```
if (a == 2)
{
    printf("Tuesday");
    break;
}
```

Case 3:

```
if (a == 3)
{
    printf("Wednesday");
    break;
}
```

Case 4:

```
printf("Thursday");
break;
}
```

Case 5:

```
printf("Friday");
break;
}
```

Case 6:

```
printf("Saturday");
break;
}
```

Case 7:

```
printf("Sunday");
break;
}
```

default:

```
{
    printf("Error");
    break;
}
```

```
{
    return 0;
}
```

loop: It is a compound statement which is used when a code needs to be repeated 'n' no. of times.

for, while, do-while.

Ques what is difference b/w while and do while?

Ques What do you mean by entry control and exit control?

for Syntax: for (Var = initial Value; Var = final value, iteration)

{

====

}

for (i = 1; i <= 10, i++)

{

====

}

while Syntax: while (condition)

{

====

iteration

}

Do while:

do

{

====

} while (condition);

- # while and do-while (frequently asked)
- # entry and exit control (Viva question)
- # break and continue

- Q. WAP to find sum of n natural no's
- Q. WAP to find factorial of a no.
- Q. WAP to find sum of even no's between 1 to 100.
- Q. WAP to print Fibonacci series upto n terms:

Ans ① #include <stdio.h>

```

int main() {
    int n, s = 0;
    for (i = 0;
    printf("Enter no. of terms: ");
    scanf("%d", &n);
    for (i = 0; i <= n; i++)
    {
        s = s + i;
    }
    printf("The sum is %d", s);
    return 0;
}

```

→ To print 'n' numbers:

// INCREMENT
LOOP

```
#include <stdio.h>
```

```
int main() { int n; i;
```

```
printf("Enter your number: ");
```

```
scanf("%d", &n);
```

```
for (i=0; i<=20, i++)
```

```
{
```

```
printf("%d\n", i);
```

```
}
```

```
return 0;
```

```
}
```

To reverse the sequence:

// DECREMENT

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int i, n;
```

```
printf("Enter your no. ");
```

```
scanf("%d", &n);
```

```
for (i=n; i>=1; i--)
```

```
{
```

```
printf("%d\n", i);
```

```
}
```

```
return 0;
```

```
}
```

Ans 2

#include <stdio.h>

int main() {

int x, i, m = 1;

printf("Enter the no. whose fac. is required :");

scanf("%d", &x);

for (i = 1; i <= x; i++) → Increment

{ m = m * i;

}

printf("your fac. is : %d", m);

return 0;

}

with do-while:

#include <stdio.h>

int main()

→ int i = 1, n, f = 1;

{

printf("Enter your no. :");

scanf("%d", &n);

do {

f = f * i;

i++;

} while (i <= n);

printf("your factorial is : %d", f);

return 0;

}

with while;

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i=1, n, f=1;
```

```
    printf("Enter your number: ");
```

```
    scanf("%d", &n);
```

```
    while (i <= n)
```

```
    {
```

```
        f = f * i;
```

```
    };
```

```
    printf("your factorial is %d", f);
```

```
    return 0;
```

```
}
```

Ans 3

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, i=1;
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &a);
```

```
    for (i=1; i <= a; i++)
```

```
    {
```

```
        if (i % 2 == 0)
```

```
        {
```

```
            i = i + 1;
```

```
        }
```

```
    printf("The sum is %d", i);
```

```
    return 0;
```

```
}
```

Ques (W)

```

#include <stdio.h>
int main
{
    int a=0, b=1, c, h, i;
    printf("Enter no. of terms");
    scanf("%d", &n);
    for (i=1; i<=n; i++)
    {
        c=a+b;
        printf("%d", c);
        a=b;
        b=c;
    }
    return 0;
}

```

Break: This statement immediately terminates the loop or switch move the execution to the statement after it.

```

int main()
{
    int i;
    for (i=1; i<=10; i++)
    {
        if (i==5)
            break;
        printf("%d", i);
    }
    return 0;
}

```

// output: 1,2,3,4

Continue: This statement skips the remaining statement of current iteration and moves to the next iteration of the loop.

last program : outputs 1,2,3,4,6,7,8,9,10

{ continue }
on 5.

goto: goto transfers program control directly to a label within the same function.

Syntax

```
goto label;
label:
statement; --
```

```
int main()
{
    int i=1;
start:
    printf("%d", i);
    i++;
    if (i <= 5)
        goto start;
    return 0;
}
```

Armstrong: $153 = 1^2 + 5^2 + 3^2$

Page No.

Date: | |

Q. WAP to find prime nos upto 50.

Q. WAP to check if the no is armstrong no or not.

Q. WAP to check Palindrome no.

(5) [121 21]

Ques WAP to print i.)

```

      x
    x x
  x x x
x x x x
x x x x x

```

ii.) A

```

  A B
A B C
A B C D
A B C D E

```

(i.)

⇒ #include <stdio.h>

int main()

{ int i, j;

for (i=1; i<=5; i++)

{ printf(" ");

for (j=1; j<=5; j++)

{

printf("%c", j);

}

printf("\n");

}

return 0;

}

(ii.)

#include <stdio.h>

int main()

{ int i, j, K;

for (i=1; i<=4; i++)

{

for (K=1; K<=4-i; K++)

{

printf("j=1; j<=1; j++)

{

printf("*")

}

printf("\n")

return 0;

}